

Equifax Data Breach: A Case Study

1. What Happened?

The 2017 Equifax data breach stands as one of the most consequential identity theft incidents in American history because the company holds sensitive financial data such as Social Security numbers, birth dates, and addresses for roughly **147.9 million** Americans, or nearly half the country's population. The disaster unfolded as a preventable failure in three acts. First, a critical security flaw in the Apache Struts software framework was publicly disclosed on March 7, 2017, and although Equifax's internal security team ordered a fix within 48 hours, the patch was never applied to the online portal used for consumer disputes. Second, hackers easily discovered the vulnerable system on March 10, and because a security tool meant to scan for malicious network traffic had been running with an expired certificate since May 2016, Equifax was completely blind while attackers roamed its systems. Finally, the breach went unnoticed for 76 days, and Equifax waited an additional six weeks after discovery to inform the public. This series of failures ultimately led to a \$700 million settlement, criminal indictments against Chinese military hackers, and a congressional report declaring the entire incident completely preventable.

2. How Did It Happen?

The Equifax data breach happened because of a chain of ignored warnings and technical oversights. On **March 7, 2017**, Apache disclosed a critical software flaw (**CVE-2017-5638**) and released a patch, followed by an urgent alert from the Department of Homeland Security. Although Equifax's internal security team instructed employees to install the patch within 48 hours, it was never applied to the company's online dispute portal. Just days later, on March 10, hackers probed the systems and easily found the open door. Compounding this failure, a security tool meant to monitor encrypted network traffic had been running with an expired certificate since May 2016, leaving Equifax completely blind to the attackers moving through their network.

3. What Was the Vulnerability?

The vulnerability that caused the 2017 Equifax data breach was a critical security flaw in Apache Struts, a software component widely used for building enterprise web applications.

- **The Flaw (CVE-2017-5638):** Publicly disclosed by Apache on March 7, 2017, this vulnerability allowed unauthorized attackers to remotely execute commands on any unpatched system.

- The Failure: Although Apache and the Department of Homeland Security immediately released patches and issued warnings, Equifax failed to apply the patch to its online consumer dispute portal (the Automated Consumer Interview System), leaving a wide open door for attackers to probe and exploit just days later.

4. Who Was Behind It?

On February 10, 2020, the U.S. Department of Justice indicted four members of **China’s People’s Liberation Army (PLA)** Wu Zhiyong, Wang Qian, Xu Ke, and Liu Lei (all from PLA Unit 54398)for carrying out the Equifax hack.

Key details regarding who was behind the attack include:

- State-Sponsored Operation: Investigators concluded that the hack was not financially motivated (as the stolen data never appeared on dark web markets), but was instead a state-sponsored intelligence operation.
- Strategic Intent: Security analysts and U.S. officials believe the massive collection of personal data was intended to build detailed dossiers on U.S. government employees, military personnel, and intelligence officers, potentially feeding into foreign intelligence and artificial intelligence tools.

5.What Was Stolen?

Data Type	Number of Records	Population Affected
Names, addresses, dates of birth, SSNs, driver’s license numbers	~147.9 million	U.S. residents
Credit card numbers	~209,000	U.S. consumers (primarily those who paid Equifax directly)
Dispute documents with personal identifiers	~182,000	U.S. consumers
Personal records (names, partial	~15.2 million	U.K. residents

financial data)		
Personal records	~19,000	Canadian residents

6. What Happened Next?

Following the 2017 data breach, Equifax faced a massive wave of fallout, legal consequences, and operational changes:

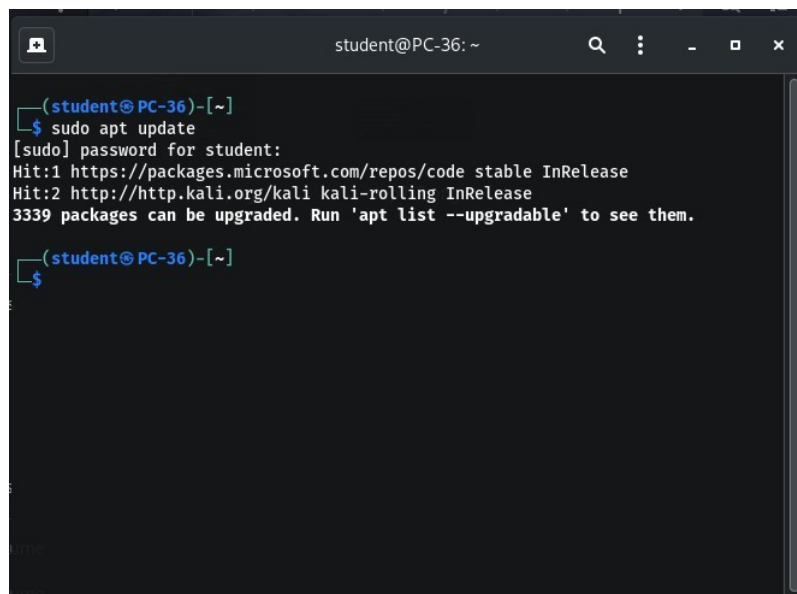
- **Discovery and Delay:** Equifax discovered the breach internally on July 29, 2017, after finally renewing an expired network security certificate. However, the company waited six weeks before publicly announcing the breach on September 7, 2017.
- **The \$700 Million Settlement:** In July 2019, Equifax reached a massive global settlement with the FTC, the CFPB, and all 50 states worth up to \$700 million. This included a consumer restitution fund, state penalties, and civil fines. Affected consumers were given options for free credit monitoring, cash reimbursement for losses, and compensation for time spent dealing with the breach.
- **Criminal Indictments:** On February 10, 2020, the U.S. Department of Justice indicted four members of China's People's Liberation Army (PLA Unit 54398) for orchestrating the hack.
- **Congressional Investigation:** A House Committee on Oversight and Government Reform investigation concluded in December 2018, formally declaring the breach entirely preventable due to Equifax's failure to apply patches and maintain adequate security protocols.

Install Docker in Kali Linux

Step 1:

```
sudo apt update
```

Output

A terminal window titled 'student@PC-36: ~' showing the output of the 'sudo apt update' command. The output indicates that two new repositories were added: 'stable' from Microsoft and 'kali-rolling' from Kali Linux. It also states that 3339 packages can be upgraded.

```
(student@PC-36)-[~]  
$ sudo apt update  
[sudo] password for student:  
Hit:1 https://packages.microsoft.com/repos/code stable InRelease  
Hit:2 http://http.kali.org/kali kali-rolling InRelease  
3339 packages can be upgraded. Run 'apt list --upgradable' to see them.  
(student@PC-36)-[~]  
$
```

Docker.io

Step 2:

```
sudo apt install -y docker.io
```

Output

```

(student@PC-36)-[~]
$ sudo apt install -y docker.io
Upgrading:
docker-cli
Installing:
docker.io
Installing dependencies:
containerd runc
Suggested packages:
containernetworking-plugins  debootstrap  xfsprogs
docker-doc  rinse  zfs-fuse
btrfs-progs  rootlesskit  | zfsutils-linux
Summary:
  Upgrading: 1, Installing: 3, Removing: 0, Not Upgrading: 3339
  Download size: 27.1 MB / 62.4 MB
  Space needed: 224 MB / 44.1 GB available
Get:1 http://http.kali.org/kali kali-rolling/main amd64 docker-cli amd64 28.5.2+dfsg4-3 [8,025 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 docker.io amd64 28.5.2+dfsg4-3 [19.1 MB]
Fetched 27.1 MB in 11s (2,427 kB/s)
Preconfiguring packages ...
Selecting previously unselected package runc.
(Reading database... 818517 files and directories currently installed.)
Preparing to unpack .../runc_1.3.5+ds1-1_amd64.deb...
Unpacking runc (1.3.5+ds1-1)...
Selecting previously unselected package containerd.
Preparing to unpack .../containerd_2.1.9+ds1-1_amd64.deb...
Unpacking containerd (2.1.9+ds1-1)...
Preparing to unpack .../docker-cli_28.5.2+dfsg4-3_amd64.deb...
Unpacking docker-cli (28.5.2+dfsg4-3) over (27.5.1+dfsg4-1)...
Selecting previously unselected package docker.io.
Preparing to unpack .../docker.io_28.5.2+dfsg4-3_amd64.deb...
Unpacking docker.io (28.5.2+dfsg4-3)...
Setting up docker-cli (28.5.2+dfsg4-3)...
Setting up runc (1.3.5+ds1-1)...
Setting up containerd (2.1.9+ds1-1)...
containerd.service is a disabled or a static unit not running, not starting it.
Setting up docker.io (28.5.2+dfsg4-3)...
Processing triggers for kali-menu (2026.1.5)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes...
Scanning processor microcode...
Scanning linux images...

Running kernel seems to be up-to-date.
The processor microcode seems to be up-to-date.
No services need to be restarted.

The processor microcode seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.

(student@PC-36)-[~]
$

```

Step 3:

```
sudo systemctl enable --now docker
```

Output

```
(student@PC-36)-[~]
$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

(student@PC-36)-[~]
$
```

Step 4:

```
sudo docker run --rm -p 80:80 vanuraalities/web-server
or
sudo docker run -rm -e n80:80 nginx
```

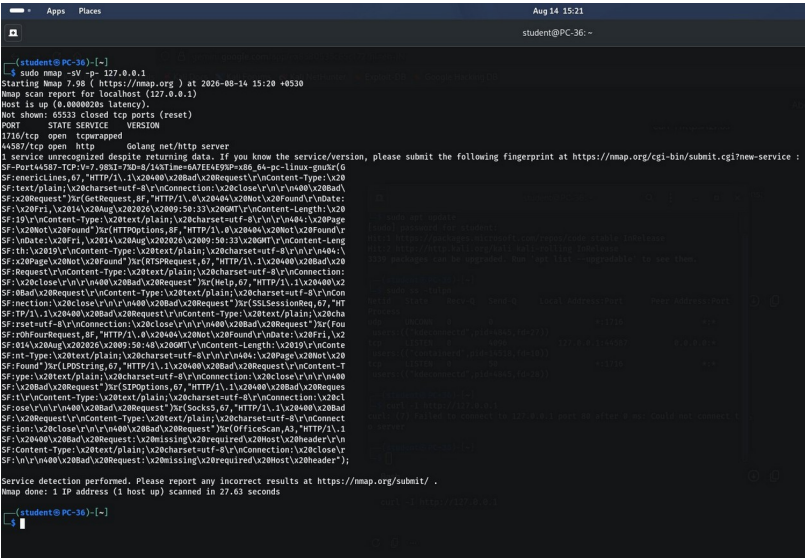
Output

```
(student@PC-36)-[~]
$ sudo docker run --rm -p 80:80 nginx
Welcome to nginx!
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/14 09:37:19 [notice] 1#1: using the "epoll" event method
2026/08/14 09:37:19 [notice] 1#1: nginx/1.31.3
2026/08/14 09:37:19 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/14 09:37:19 [notice] 1#1: OS: Linux 6.18.12-kali-amd64
2026/08/14 09:37:19 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/08/14 09:37:19 [notice] 1#1: start worker processes
2026/08/14 09:37:19 [notice] 1#1: start worker process 28
2026/08/14 09:37:19 [notice] 1#1: start worker process 29
2026/08/14 09:37:19 [notice] 1#1: start worker process 30
2026/08/14 09:37:19 [notice] 1#1: start worker process 31
2026/08/14 09:37:19 [notice] 1#1: start worker process 32
2026/08/14 09:37:19 [notice] 1#1: start worker process 33
2026/08/14 09:37:19 [notice] 1#1: start worker process 34
2026/08/14 09:37:19 [notice] 1#1: start worker process 35
2026/08/14 09:37:19 [notice] 1#1: start worker process 36
2026/08/14 09:37:19 [notice] 1#1: start worker process 37
2026/08/14 09:37:19 [notice] 1#1: start worker process 38
2026/08/14 09:37:19 [notice] 1#1: start worker process 39
2026/08/14 09:37:19 [notice] 1#1: start worker process 40
2026/08/14 09:37:19 [notice] 1#1: start worker process 41
2026/08/14 09:37:19 [notice] 1#1: start worker process 42
2026/08/14 09:37:19 [notice] 1#1: start worker process 43
2026/08/14 09:37:19 [notice] 1#1: start worker process 44
2026/08/14 09:37:19 [notice] 1#1: start worker process 45
2026/08/14 09:37:19 [notice] 1#1: start worker process 46
2026/08/14 09:37:19 [notice] 1#1: start worker process 47
2026/08/14 09:37:19 [notice] 1#1: start worker process 48
2026/08/14 09:37:19 [notice] 1#1: start worker process 49
2026/08/14 09:37:19 [notice] 1#1: start worker process 50
2026/08/14 09:37:19 [notice] 1#1: start worker process 51
2026/08/14 09:37:19 [notice] 1#1: start worker process 52
2026/08/14 09:37:19 [notice] 1#1: start worker process 53
2026/08/14 09:37:19 [notice] 1#1: start worker process 54
2026/08/14 09:37:19 [notice] 1#1: start worker process 55
172.17.0.1 - - [14/Aug/2026:09:38:14 +0000] "GET / HTTP/1.1" 200 896 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" "-"
172.17.0.1 - - [14/Aug/2026:09:38:14 +0000] "GET /favicon.ico HTTP/1.1" 404 153 "http://127.0.0.1/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0" "-"
2026/08/14 09:38:14 [error] 28828: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 172.17.0.1, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "127.0.0.1", referer: "http://127.0.0.1/"
172.17.0.1 - - [14/Aug/2026:09:38:26 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.18.0" "-"
```

Step 5:

```
sudo nmap -sV -p- 127.0.0.1
```

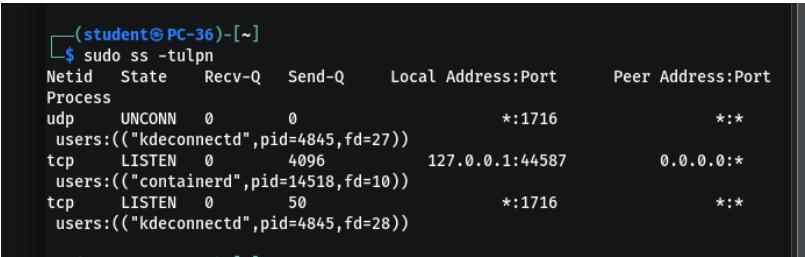
Output



Step 6:

```
sudo ss -tulpn
```

Output



Step 7:

```
curl -I http://127.0.0.1
```

Output

```
(student@PC-36)-[~]  
$ curl -I http://127.0.0.1  
HTTP/1.1 200 OK  
Server: nginx/1.31.3  
Date: Fri, 14 Aug 2026 09:51:57 GMT  
Content-Type: text/html  
Content-Length: 896  
Last-Modified: Wed, 15 Jul 2026 16:03:14 GMT  
Connection: keep-alive  
ETag: "6a57af42-380"  
Accept-Ranges: bytes
```

```
(student@PC-36)-[~]  
$
```
